27/03/2026

Sistema de Gestión de Biblioteca Digital

Programación Orientada a Objetos

Christian Cabo Terriza

1. POO AVANZADA

He organizado el proyecto en módulos separados:

- dominio → Contiene Libro, Usuario, Prestamo, Biblioteca.
- servicio → Contiene validadores, multas, exportador, plugins.
- plugin → Contiene los plugins de: estadísticas y exportación PDF.
- main → Interfaz de consola.
- exportacion → Directorio donde se guardan los archivos CSV y JSON generados.
- importacion → Directorio donde se cargan los archivos CSV y JSON .
- test → Directorio destinado a pruebas del sistema.

He diseñado el sistema usando la biblioteca como agregado raíz del sistema. Los módulos de libros, usuarios y préstamos funcionan como entidades simples, y he decidido calcular los estados en lugar de guardarlos. Además, he mantenido una separación estricta entre las capas de dominio e infraestructura.

2. USO DE HERENCIA MÚLTIPLE Y AL MENOS UN MIXIN

He utilizado mixins para separar funcionalidades, primero he creado ExportadorMixin lo he utilizado en exportador, esto permite exportar datos a CSV y JSON sin que forme parte del dominio y para volverlo a usar en la clase biblioteca.

Por otro lado, también tengo ExportarPDFMixin, el cual he utilizado en exportar_pdf_plugin, lo que permite añadir funcionalidad sin modificar la clase principal, además de integrarse mediante herencia múltiple.

3. IMPLEMENTACIÓN DE DECORADORES PARA VALIDACIONES

Los decoradores los he implementado en el módulo validadores.

La función de estos decoradores es validar datos antes de ejecutar métodos, comprobar los parámetros obligatorios y evitar errores en operaciones de la biblioteca. Por ello, este diseño da la ventaja de centralizar la validación y evitar duplicar código, también mantiene el dominio libre de validaciones relacionadas con la entrada del usuario.

4. APLICACIÓN DE UN PATRÓN DE DISEÑO

He implementado el patrón Strategy para el cálculo de multas en el archivo multas.

Empecé con MultaStrategy, que es la única clase abstracta del programa, la cual decidí que sea abstracta, para que, de esa manera, pueda variar el método cálculo de la multa y, de esta forma, poder cambiar el método sin modificar el resto del código.

Y después continué con los dos tipos de métodos de cálculo de multas: MultaPorDia, que multiplica la cantidad de dinero ingresada y los días de retraso para calcular el total a pagar, y MultaFija, que como indica el nombre, es un precio fijo.

5. USO DE MÉTODOS ESPECIALES

He decidido implementar estos métodos especiales para darle una mayor funcionalidad a esta parte del código:

- __str__ en libros , usuarios, préstamos.
- __repr__ para facilitar la depuración en libros, usuarios, préstamos.
- __iter__ en Biblioteca
- __getitem__ en Biblioteca

Con esto se consigue una representación clara de los objetos y permite el acceso a sus elementos mediante una interfaz de diccionario.

6. Uso de metaclases

He usado la metaclase MetaclaseEntidad en los módulos de libros, usuarios y préstamos. Con ella, valido la estructura de las clases del dominio y compruebo que se definan todos los métodos necesarios, lo que me permite detectar errores. He decidido encargar la validación estructural a esta metaclase.

7. CONCLUSIÓN

A modo de recopilación, estos son todos los elementos que he integrado para estructurar mi propuesta final:

- Mixins para separar funcionalidades.
- Strategy para comportamiento variable.
- Decoradores para validación.
- Metaclases para control estructural.
- Métodos especiales para mejorar el uso de objetos.
- Plugins para extender el sistema.

Mi prioridad no ha sido únicamente que el código funcione correctamente, sino también que el diseño tenga sentido a nivel estructural. He buscado que sea coherente, modular y fácil de entender, de manera que cada parte del sistema tenga una responsabilidad clara. Esto mejora la organización y mantenimiento del proyecto. Además, considero importante que el código sea legible y reutilizable, ya que esto permite trabajar sobre él de forma más cómoda y sostenible a largo plazo.